

Spare Register Aware Prefetching for Graph Algorithms on GPUs

Nagesh B. Lakshminarayana Hyesoon Kim
School of Computer Science
Georgia Institute of Technology
{nagesh.bl, hyesoon.kim}@gatech.edu

Abstract

More and more graph algorithms are being GPU enabled. Graph algorithm implementations on GPUs have irregular control flow and are memory-intensive with many irregular/data-dependent memory accesses. Due to these factors graph algorithms on GPUs have low execution efficiency. In this work we propose a mechanism to improve the execution efficiency of graph algorithms by improving their memory access latency tolerance. We propose a mechanism for prefetching data for load pairs that have one load dependent on the other - such pairs are common in graph algorithms. Our mechanism detects the target loads in hardware and injects instructions into the pipeline to prefetch data into spare registers that are not being used by any active threads. By prefetching data into registers, early eviction of prefetched data can be eliminated. We also propose a mechanism that uses the compiler to identify the target loads. Our mechanism improves performance over no prefetching by 10% on average and upto 51% for nine memory intensive graph algorithm kernels.

1. Introduction

More and more algorithms and applications are being GPU enabled, not only those that are embarrassingly parallel, but those that are unstructured/irregular and data intensive as well. Considering energy and power efficiency, GPU is an attractive solution for handling such parallel and data intensive applications. With graphs being used in numerous applications or application domains like social computing, bio-informatics, communication networks, network security, navigation systems, search engines, scheduling in transportation networks, it is imperative for graph algorithms to execute efficiently on GPUs if GPUs are to be used for irregular and data intensive applications. And thus there has been considerable recent work enabling and improving the performance of fundamental graph algorithms such as Breadth First Search, Minimum Spanning Tree and Single Source Shortest Path and others on GPUs [20, 47, 21, 43, 23, 22, 32, 5]. However, the irregular and data intensive nature of graph algorithms presents several obstacles to the GPU.

Graph algorithms and other irregular applications tend

to have low execution efficiency on GPUs. The reasons are primarily three, namely, control flow divergence, non-uniform work distribution and large number of memory accesses with divergence. Because of the large number of memory accesses that tend to be irregular/data-dependent, graph algorithms are often unable to hide memory access latency. Significant work has already been done on reducing the negative performance impact of control flow divergence [16, 31, 15, 12, 37, 4] in GPGPU applications. However, the other two problems have been much less explored.

Low utilization of GPUs reduces the performance benefits of using GPUs, but it also provides us with new opportunities for innovation. Unlike CPU cores which aggressively use all the available architecture/physical registers, GPU cores could have several unused physical registers. The physical register file in GPUs is designed to support a massive number of threads, but often applications do not utilize all available registers. Figure 1 shows the size of unused registers out of a 128KB register file for several different graph algorithm kernels (see Table 4 for kernels).¹ As we can see, a significant fraction of the register file is unused. Furthermore, in the case of irregular applications, the number of unused registers can vary at run-time because of the irregularity of applications. Hence, in this paper, we propose to utilize the unused registers in GPUs in coordination with prefetching to attack problems due to memory accesses in graph algorithm.

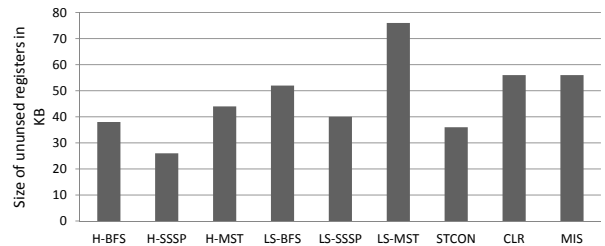


Figure 1: Unused registers out of 128KB register file

Prior prefetching mechanisms use caches to store prefetched data. Instead, the proposed mechanism stores prefetched data in unallocated spare registers that are available in a GPU core. By doing so, our mechanism escapes

¹NVIDIA's Fermi architecture uses a 128KB register file per GPU core

from the problem of early eviction of prefetched data placed in the cache. Early eviction is a much more serious problem in GPU architectures than in CPUs due to the large number of threads and the relatively small cache size available per thread. Binding software prefetching also utilizes register files to store prefetched data but binding prefetch instructions could cause high overhead to check legitimacy of prefetched memory addresses. Furthermore, our mechanism is specialized to prefetch dependent loads that are the key memory access patterns in graph algorithms. We elaborate on the differences between our proposal and prior mechanisms in Section 2.4 and Section 6. In summary, this paper makes the following contributions:

1. We show that irregular GPGPU applications tend to have large spare (unused) register files at run-time. Hence, we propose a prefetching mechanism to store prefetched data in spare registers to reduce prefetch overhead in caches such as early eviction.
2. We propose a hardware-only mechanism for detecting and prefetching load-pairs with one load dependent on the other to benefit graph algorithms on GPUs.

2. Background

2.1. GPGPU terminology

We use terminology that is consistent with NVIDIA's CUDA [36] : a warp is group of 32 threads that execute in lock-step, a thread block is the granularity at which work is assigned to GPU cores called Streaming Multiprocessors (SMs), a block consists of one or more warps, threads in the same block can synchronize using barriers and a kernel consists of one or more blocks.

2.2. Graph Algorithms on GPUs

Graph algorithms and hence their implementations are iterative with multiple steps in each iteration. There is data dependency between successive iterations and between the steps of an individual iteration. The data dependency between the steps of an iteration means synchronization is required between the steps and hence on GPUs each step is often implemented as one or more kernels with a kernel launched only when the previous one has terminated.

2.2.1 Graph Representation on GPUs

A common way for representing graphs in CUDA implementations is the compact adjacency list. In this representation, a graph $G(V, E)$ is represented using essentially two lists (arrays) - the Vertex list and the Edge list. The vertex list has one element for each vertex in the graph with the element for vertex v being stored at index v in the list. The edge list contains the destination vertex ids for all the edges in the graph. For a given vertex v , the endpoints of all

edges from v are stored contiguously in the edge list. The element for v in the vertex list contains the index of the location in the edge list where the endpoint of the first edge from v is stored. The number of edges originating from v can be computed as $(vertex_list[v + 1] - vertex_list[v])$. Compared to an adjacency matrix, a compact adjacency list consumes less space to store sparse graphs and also make neighbor traversal easier. Figure 2 shows an example graph in compact adjacency list form.

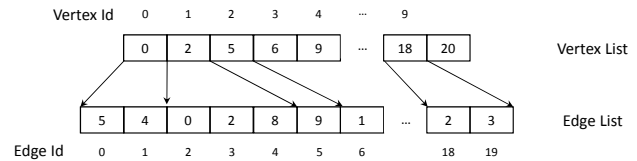


Figure 2: Graph in Compressed Adjacency List form

2.3. Spare Registers in GPUs

Unlike CPUs which utilize all available physical registers, a GPU could have several unused physical registers. There are two sources of spare registers in a GPU:

1. Due to resource constraints there may be unallocated registers in a GPU. Threads are assigned to SMs at thread block granularity i.e., only if the resource requirement for all threads in a block can be satisfied will a block be assigned to a SM. It often happens that not enough resources are available to allocate an entire additional block, but a few additional registers can be assigned to each of the already assigned threads without reducing the occupancy of the kernel (see Table 4).
2. If a thread warp has terminated, then registers allocated to the warp can be used by other warps until the thread block containing the terminated warp has terminated and a new block is assigned to the SM.

2.4. Prefetching for graph algorithms

```

Load-A → for(int i = start; i < (start + no_of_edges); i++) {
Load-B →   int id = g_graph_edges[i];
           if(!g_graph_visited[id]) {
           ...
           }
           }

```

Figure 3: Example code

2.4.1 Sample Code

We refer to the code in Figure 3 which is taken from an implementation of Breadth First Search (BFS) [20] and shows the code for processing a vertex assigned to a thread. The neighbor list of the vertex starts at index $start$ in array g_graph_edges and ends at index $(start + no_of_edges)$

- 1). The thread accesses each neighbor of the vertex and some information about the neighbor; such processing can be considered to be typical of several graph algorithms on GPUs. Load-A loads from memory the vertex id of each neighbor and Load-B loads the flag which indicates whether a neighbor has already been visited by the search or not. Here, the index into the array *g_graph_visited* is loaded from array *g_graph_edges*, thus Load-B is dependent on Load-A. The accesses to *g_graph_edges* are sequential/small strided across different loop iterations and thus have spatial locality. Depending on the input graph, accesses to *g_graph_visited* are unlikely to have any locality and a significant fraction of the execution time is spent on waiting for loads from *g_graph_visited* to complete.² Our proposal is in each loop iteration prefetch values from *g_graph_edges* and *g_graph_visited* that would be accessed in a subsequent iteration.

2.4.2 Hardware Prefetching

In CPUs, hardware Prefetchers are trained with the last level cache (LLC) miss address stream. With sufficient training, the prefetchers can detect patterns or correlations in the miss address stream and can predict future miss addresses. The prefetcher designs used in CPUs cannot be directly applied to GPUs [27] - the prefetchers have to be thread (or warp) aware. In addition, for benchmarks with irregular and data dependent memory accesses, training using the LLC miss address stream will not help in detecting correlation in the memory accesses.

For the access pattern in Figure 3, a thread-aware hardware prefetcher that learns using the LLC miss address stream may detect that Load-A has strided accesses. If cache block addresses are used for training it is likely that no stride will be detected since Load-A is small strided and all accesses by a thread may be to the same cache block. Since Load-B does not have any strided behavior, a thread-aware prefetcher will be unable to prefetch the addresses accessed by Load-B. Also, Load-B is likely to have more cache misses than Load-A, and a thread-aware hardware prefetcher that does not prefetch Load-B may not show much benefit. Even complex prefetchers such as a Markov prefetcher [26] or a Content-Directed Data prefetcher (CDP) [10] would be unable to detect the relation between Load-A and Load-B.³

2.4.3 Software Prefetching

For the access pattern in Figure 3 software prefetching can be performed in several different ways:

² multi-threading is not sufficient to hide memory access latency for the benchmarks considered

³ A CDP looks for virtual addresses in the cache blocks and in this case the blocks only contain the index of the edge element to be accessed.

Prefetching Load-A only Load-A can be prefetched into either the data cache or into a register. This would not result in significant performance benefit, since Load-B is the performance critical load.

Prefetching Load-B into cache Due to the dependency of Load-B on it, Load-A must be prefetched into a register to prefetch Load-B i.e., a binding prefetch must be issued for Load-A to prefetch Load-B. However, prefetching Load-B into the cache could increase memory accesses due to eviction of prefetched data and could hurt performance.

Binding prefetches for both Load-A and Load-B This approach is similar to the mechanism we propose (Section 3) with some subtle differences. Firstly, software prefetching is an all or nothing approach. For software prefetching to be performed, sufficient free registers must be available to do binding prefetching for all threads of all blocks that can be assigned to a SM at a given time, otherwise occupancy will be reduced. While software prefetching reduces occupancy if insufficient registers are available, our mechanism can be adapted to do prefetching for only a subset of the warps that will be allocated to a SM without reducing occupancy. However, we do not evaluate such an extension to our mechanism in this paper. Also, software prefetching uses a fixed prefetch distance, while our mechanism can adapt the prefetching distance dynamically on a per loop invocation basis for each warp.

2.5. Loop Unrolling for graph algorithms

Loop unrolling with load hoisting could also be performed for the access pattern in Figure 3. However, the degree of unrolling should be determined statically. Furthermore, all threads in a kernel will have the same degree of unrolling because they all execute the same static code. Also, since loop unrolling replicates the entire loop body, it is likely to increase register usage and could reduce occupancy. As explained, the proposed mechanism can vary the prefetch distance dynamically and can be easily modified to prefetch for only a subset of the warps without reducing occupancy if the number of spare registers are not sufficient.

3. Mechanism

The proposed mechanism is entirely based in hardware and is explained in Sections 3.2.1- 3.2.5, while a compiler-assisted mechanism is explained in Section 3.2.6.

3.1. Overview

We first identify a target loop and the target loads within the loop. The target loads usually occur in pairs - an independent load with strided accesses (similar to Load-A in Figure 3) and a load dependent on the independent load (similar to Load-B in Figure 3). We refer to a pair of loads with one independent load and a dependent load as a load-pair. The independent load is called the head-load and the dependent load is called the tail-load.

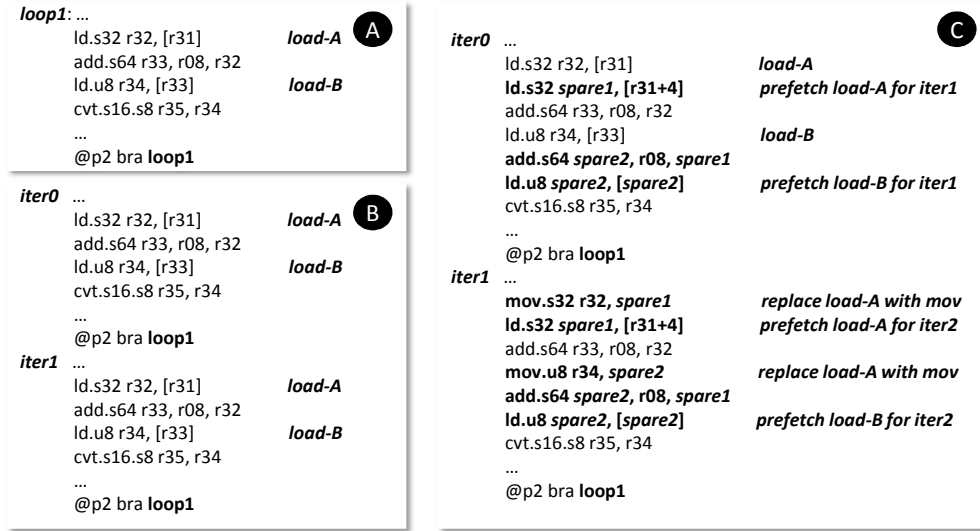


Figure 4: A. Example loop B. Regular instruction stream C. Instruction stream with proposed mechanism

In each loop iteration we inject instructions to prefetch data that will be accessed by the target loads in subsequent loop iterations. The injected instructions prefetch data into available spare registers (as well as cache). And the load instructions whose addresses have been prefetched are translated into mov instructions that copy the prefetched data value into the destination register of the original load instruction. The mechanism is designed to ensure that the prefetch addresses are correct, therefore we can prefetch data into registers and use the prefetched data. The injection of the prefetching instructions can be explained with the help of the code in Figure 4A which is the PTX code for the example in Figure 3. This injection process could also be thought of as doing software pipelining in hardware - in each iteration we inject loads that are used by subsequent iterations. However, the loop does not have any prologue and epilogue.

The instruction stream under normal execution is shown in Figure 4B and the instruction stream with the proposed mechanism is shown in Figure 4C. In the original instruction stream both Load-A and Load-B are used immediately after being fetched. For the applications considered, load wait times are significant, especially for uncoalesced loads such as Load-A and Load-B and multi-threading is unable to hide memory access latency fully. However, for the instruction stream with prefetching, the wait time for Load-B would be greatly reduced since it is prefetched one iteration before it is used. Since Load-A is strided with a small stride, accesses generated by Load-A in successive iterations for a thread are more likely to be cache hits in either L1 or L2 cache than accesses generated by Load-B. Overall, the wait for Load-A may not be as significant as the wait for Load-B and reducing the wait time for Load-B helps improve performance more. And using registers for storing prefetched

data eliminates the problem of eviction that would plague mechanisms that insert prefetched data into the cache only.

3.2. Details

Before injecting instructions for prefetching data the mechanism has to identify the target loads for prefetching.

3.2.1 Identifying target loads for prefetching

The first step in identifying target loads is to detect loops. A loop is detected when a backward branch is encountered by a warp. The branch PC is the end of the loop and the branch target is the start address of the loop. To identify the target loads within a loop we used a tag-based hardware mechanism. Our approach uses two sets of tags - *value* tags to identify whether loads are strided or otherwise; *load-id* tags to detect dependencies between loads. There is one set of tags of each type per SM (not per warp) with one tag for each architecture register; each tag is only 2-bits wide. Figure 5 shows an example loop and how the load properties are detected.

Once a loop is detected, the instruction decoder selects a warp that is beginning a new loop iteration for identifying the target loads (if any) in the loop. At the start of the loop iteration of the selected warp, the *value* tags of all registers are set to *constant*. The tags of the destination register ids for instructions that are commonly used in address generation are updated as shown in Table 1 (update rules for other instructions are not shown). At the end of the loop when the loop index variable is updated, a WAR update (register is read and written in the same instruction) with a constant increment is detected and the *value* tag of the register id holding the loop index variable is updated as *stride* (Figure 5B). When the next loop iteration is commenced, the updated tag values are used and we are able to detect whether a load is

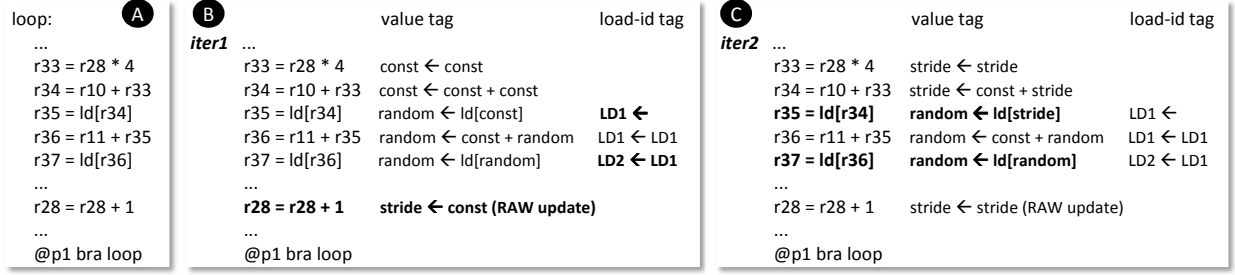


Figure 5: Detecting target loads and their properties

strided or not by the end of the third loop iteration (one iteration for detecting loop and two iterations for detecting load properties) as shown in Figure 5C.

The *load-id* tags are used to track dependences between loads. When a value is loaded into a register from memory, the register is assigned a *load-id* tag and the tag is propagated. If the source of a load has a valid *load-id* tag then the load is dependent on the load whose tag is in the source tag.

The tag-based mechanism has to ensure that the prefetched addresses will be always correct since we are prefetching data into registers and using it.

Table 1: Tag Propagation

Dest Tag	Opcode	Op-0 RAW update?	Op-0 Tag	Op-1 Tag
random	ld	X	X	X
const	add/sub/mul/shl	0	const	const
stride	add/sub/mul/shl	0	stride	const
stride	add/sub/mul/shl	1	const/stride	const

3.2.2 Spare registers for prefetch destinations

The two sources of spare registers were explained in Section 2.3 and in this work we only consider unallocated registers for holding prefetch data. Table 4 shows the number of registers required by each thread (MaxReg_s) for several graph algorithm kernels and also the maximum number of registers (MaxReg_d) that can be allocated to each thread without reducing the occupancy of the kernel. When allocating registers, we allocate MaxReg_d registers to each thread, instead of allocating only MaxReg_s registers.

3.2.3 Prefetch distance

In the proposed mechanism prefetch distance refers to how many iterations in advance are prefetches inserted. Either a fixed prefetch distance could be used for all loop invocations or the distance could be varied for each invocation.

Fixed prefetch distance If sufficient registers are available, the simplest form of our mechanism inserts prefetches with a distance of one. To prefetch with a distance of *pref_dist*, for each load we need *pref_dist* number of spare registers to hold the prefetched value(s).

Variable prefetch distance The prefetch distance could be varied per loop invocation depending on the average

number of loop iterations per thread as shown in Algorithm 1.⁴ A larger prefetch distance potentially reduces the wait time for the tail-load. Section 3.2.4 explains the calculation of the number of loop iterations of each thread.

Algorithm 1 Calculating the prefetch distance

```

if avg_iter_count < MID then
  pref_distance = low
else if avg_iter_count < HIGH then
  pref_distance = med
else
  pref_distance = high
end if

```

If the spare registers required by the calculated prefetch distance are not available, the prefetch distance is reduced until the required number of spare registers are available.

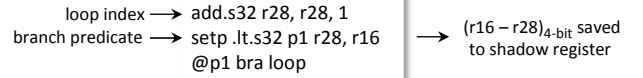


Figure 6: Determining iteration counts for each thread

3.2.4 Determining number of loop iterations

We use a shadow register of 4-bits per thread for each warp to hold the iteration count for each thread. In GPUs, conditional branches are implemented as predicated jump instructions. For the target loop, we detect the *setp* instruction that sets the predicate register on which the loop-ending branch is predicated. The *setp* instruction usually evaluates a "less than" condition with the current value of the loop index and the final value of the loop index as its operands. When the *setp* instruction is evaluated, we also evaluate a modified *sub* operation with the same operands as the *setp* instruction to obtain the pending iteration count for each active thread in the warp as shown in Figure 6. The average number of loop iterations per active thread can be calculated from these 4-bit shadow registers. The average iteration count is calculated only once while the shadow register is updated every iteration.

⁴ we use the following values for the constants in Algorithm 1: low = 1, med = 2, high = 3, MID = 4, HIGH = 6

3.2.5 Inserting Prefetches

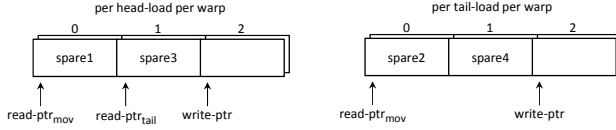


Figure 7: Tracking registers allocated for prefetches

Once the prefetch distance has been calculated by the hardware, instructions are injected into the pipeline to prefetch data into spare registers. We accomplish this using circular lists called *AllocatedReg* lists (Figure 7), which are of size three (the largest allowed prefetch distance) and hold the register ids allocated for holding prefetched data; each warp requires one list per load. The list for a head-load has three pointers: *write-ptr* which points to the next location to be used - the id of the next allocated spare register is updated in this location, *read-ptr_{mov}* which points to the next location to be read by the *mov* instructions replacing the *ld* instructions and *read-ptr_{tail}* which points to the next location to be read by the instructions inserted for prefetching the tail-loads. Each tail pointer also has a similar circular list but with only two pointers - *write-ptr* and *read-ptr_{mov}*. After inserting prefetches for *pref_{dist}* number of iterations, the head and tail loads in subsequent iterations are converted into *mov* instructions. Figure 4 shows the insertion of head and tail prefetches and the conversion of loads into *mov* instructions. Spurious prefetch instructions (when threads have exited the loop) are avoided by using the shadow registers containing the iteration counts to control the generation of prefetch instructions and their active masks.

3.2.6 Compiler-assisted mechanism

The hardware-only mechanism requires three non-overlapping loop iterations (the iterations need not be from the same warp) to detect the target loads. Due to the time overhead of this mechanism we are unable to insert prefetches for a few iterations for the initial set of warps assigned to each SM. In addition, to eliminate spurious prefetch memory loads, the hardware-only mechanism needs to know the iteration counts of the threads before inserting prefetches, and determining iteration counts takes one iteration for each warp. Thus the hardware prefetcher misses several prefetching opportunities. Instead, if we take the aid of the compiler, we can insert prefetches from the first loop iteration of each warp. The compiler can not only identify the target loads and their properties but can also mark the instruction which tests the loop condition before the loop is entered. This instruction would be a *setp* instruction which could be used to determine loop iteration counts before entering the loop (similar to Section 3.2.4).

Thus a compiler-assisted mechanism can insert prefetches starting from the first iteration itself.

Table 2: Hardware Cost

Component	Size	Cost
Value tags	2-bits * 63	126 bits
Load-id tags	2-bits * 63	126 bits
MaxReg _s	6 bits	6 bits
MaxReg _d	6 bits	6 bits
Prefetch distance counter	2 bits * 48 warps	12 bytes
Shadow iteration count reg	4 bits * 32 threads * 48 warps	768 bytes
AllocatedReg list		
Head-load/warp	6 bits/entry * 3 + 2 bits/ptr * 3	3 bytes
Tail-load/warp	6 bits/entry * 3 + 2 bits/ptr * 2	2 bytes + 6 bits
2 load-pairs (all warps)	(11 bytes + 4 bits) * 48 warps	552 bytes
Total		1365 bytes (1.04% of reg file)

3.2.7 Total Hardware Cost

Table 3.2.6 shows the total hardware cost of the hardware-only mechanism with support for variable prefetch distances while assuming a maximum of 63 registers per thread and a maximum of 48 warps per SM in accordance with the Fermi configuration [35]. The compiler-assisted mechanism would not require the value tags and the load-id tags used for target load detection.

4. Methodology

Table 3: Simulated GPGPU Architecture

Num. of cores	16
Front End	Fetch width: 2 warp-instruction/cycle; 4KB I-cache; stall on branch; 5 cycle decode
Execution core	2 warp-instructions/cycle, Frequency: 1.2 GHz; 16-wide SIMD, in-order scheduling, latencies are modeled according to the CUDA manual [36]; 128 KB register file
On-chip caches	16/48 KB software managed cache, 16 loads/2-cycle; 4-cycle 6-way 48KB L1 cache, 16 loads/2-cycle or 4-cycle 4-way 16KB L1 cache, 16 loads/2-cycle; 4-cycle 8 KB constant cache, 16 loads/2-cycle; 4-cycle 8 KB texture cache, 16 loads/2-cycle; 100-cycle round trip access to 768 KB L2 cache;
DRAM	2 KB page, 16 banks with page interleaving, 128 reqs/bank, 8 channels, 192.4 GB/s, 1.6 GHz, 400 cycle minimum round-trip time; tCL=20, tRCD=28, tRP=12 (GDDR-5 [1])

We emulate several graph applications with different inputs (Section 4.1) using GPUOcelot [13] and collect their traces which are simulated using MacSim [2], a cycle-level heterogeneous architecture simulator. The simulated architecture, shown in Table 3 is based on NVIDIA's Fermi [34].

Table 5: Input graph characteristics

Input	#Vertices	#Edges	Avg. Degree	BFS Depth
Graph1M	1000000	5999970	5.99	12
RMAT20	1048576	8259994	7.88	13
R4_2e20	1048576	4194304	4	16

4.1. Benchmarks and inputs

We evaluate kernels from several fundamental graph algorithm implementations; the evaluated kernels and their

Table 4: Evaluated kernels

Application	Kernel name	Warps per block	Blocks per SM	Registers per Thread (actual)	Registers per Thread (max)	Unused register space
Breadth First Search [20] (H-BFS)	Kernel	16	3	15	20	38KB
Single Source Shortest Path [20](H-SSSP)	DijkstraKernel1	16	3	17	20	26KB
Minimum Spanning Tree [47] (H-MST)	Kernel_Find_Min_Edge_Weight_Per_Vertex	16	2	21	32	44KB
Breadth First Search [5] (LS-BFS)	drelax	32	1	19	32	52KB
Single Source Shortest Path [5] (LS-SSSP)	drelax	32	1	22	32	40KB
Minimum Spanning Tree [5] (LS-MST)	boruvkaone	32	1	13	32	76KB
S-T Connectivity [20] (STCON)	kernel	16	2	23	32	36KB
Graph Coloring [8] (CLR)	Kernel	16	3	12	20	56KB
Maximal Independent Set [28] (MIS)	Kernel	16	3	12	20	56KB

attributes such as register usage per thread and the maximum number of registers that can be allocated per thread without reducing occupancy are shown in Table 4.⁵ H-BFS, H-SSSP and H-MST are taken from implementations provided by Harish, Vineet et al. [20, 47, 21] and LS-BFS, LS-SSSP and LS-MST are taken from LonestarGPU 0.9 [5]. We implement ST-CON [20], Graph Coloring [8, 7] and Maximal Independent Set [28, 7] algorithms based on prior work. For each application we generate traces for multiple input graphs and since graph algorithms are iterative, each kernel is invoked multiple times during a single application run. Some characteristics of the different input graphs are shown in Table 5. Graph1M is obtained from the Parboil benchmark suite [46] and is a random graph with a average vertex degree of 6, while RMAT20 and R4_2e20 are obtained from LonestarGPU 0.9 [5]. RMAT20 represents graphs found in the real world [6], it has few vertices with a large degree and a large number of vertices with a small degree; while R4_2e20 is a random graph with a average vertex degree of 4.

5. Results

Table 6: Evaluated hardware prefetchers

Prefetcher	Description	Configuration
Stride	RPT Stride Prefetcher [24]	1024-entry, 16 region bits
Stream	Stream Prefetcher [45, 42]	512-entry
GHB	AC/DC GHB Prefetcher [25, 33]	1024-entry GHB, 12-bit CZone, 128-entry Index Table

We evaluate different forms of the proposed prefetch mechanism - hardware only with a prefetch distance of 1 (SPREF1), compiler-assisted with a prefetch distance of 1 (C-SPREF1), hardware-only with a dynamic prefetch distance (SPREF) and compiler-assisted with a dynamic prefetch distance (C-SPREF). We compare our mechanism against Stride, Stream and GHB hardware prefetchers; the configurations for these prefetchers are shown in Table 6. All prefetch mechanisms insert prefetched data into both L2 and L1 caches. Only the proposed mechanisms insert prefetched data into registers as well. Stride, Stream and GHB prefetchers are trained on L1 (and L2) access streams to improve their learning.

Since a kernel is invoked multiple times during the same application run (with a given input), for the sake of read-

ability we show results for the different kernel invocations during a application run using a single value. We show the harmonic mean of the ratio of the IPC with the evaluated mechanism to the IPC of the baseline (i.e., no prefetching) across all kernel invocations for a given input. In the bar graphs, the harmonic mean value for a give input is labeled with the name of the input graph. We also show the harmonic mean of the IPC ratio across all kernel invocations for all inputs in the bar graphs; these bars are labeled HMean in the graphs. Our evaluation considers only those kernel invocation instances that have about 1 million warp instructions or more. The IPC metric that is used is a weighted IPC metric calculated across all GPU cores. For the proposed mechanism, when calculating the IPC we ignore the additional instructions introduced by the hardware (but these additional instructions are simulated fully), thus the total instruction count for the proposed mechanism is the same as the baseline.

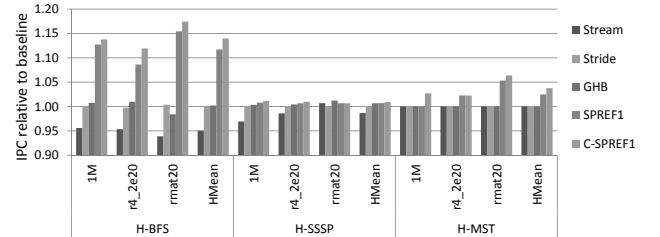


Figure 8: IPC improvements - H-BFS, H-SSSP, H-MST

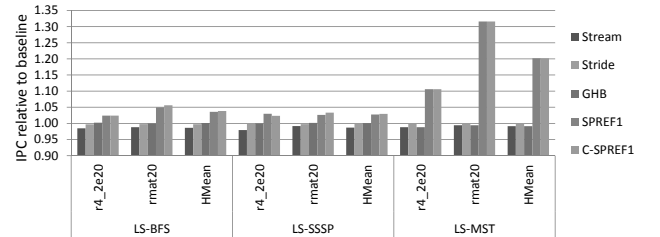


Figure 9: IPC improvements - LS-BFS, LS-SSSP, LS-MST

5.1. Performance improvement with different mechanisms

Figures 8, 9 and 10 show the performance improvement provided by Stride, Stream, GHB, SPREF1 and C-SPREF1 for the evaluated kernels with different inputs. While the

⁵We include only algorithms that have the access pattern described in Section 2.4.1

Table 7: IPC improvements over baseline (SPREF1, C-SPREF1, SPREF, and C-SPREF are proposed mechanisms); HM: Harmonic Mean of IPC improvements

	H-BFS		H-SSSP		H-MST		LS-BFS		LS-SSSP		LS-MST		STCON		CLR		MIS		
	HM	Max	HM	Max	HM	Max	HM	Max	HM	Max	HM	Max	HM	Max	HM	Max	HM	Max	HM (all)
Stream	0.95	1.07	0.99	1.03	1.00	1.00	0.99	1.00	0.99	1.01	0.99	0.99	1.00	1.05	1.00	1.01	1.00	1.01	0.99
Stride	1.00	1.02	1.00	1.01	1.00	1.00	1.00	1.00	1.00	1.02	1.00	1.00	1.00	1.01	0.96	1.00	0.95	1.00	0.98
GHB	1.00	1.06	1.01	1.06	1.00	1.00	1.00	1.01	1.00	1.02	0.99	0.99	1.02	1.07	1.00	1.02	1.00	1.02	1.00
SPREF1	1.12	1.45	1.01	1.04	1.02	1.05	1.04	1.07	1.03	1.06	1.20	1.32	1.04	1.18	1.16	1.26	1.12	1.23	1.08
C-SPREF1	1.14	1.46	1.01	1.04	1.04	1.06	1.04	1.07	1.03	1.08	1.20	1.32	1.05	1.21	1.17	1.28	1.15	1.32	1.09
SPREF	1.12	1.50	1.01	1.04	1.05	1.06	1.04	1.07	1.03	1.07	1.23	1.39	1.04	1.20	1.16	1.26	1.12	1.23	1.08
C-SPREF	1.15	1.51	1.01	1.04	1.05	1.06	1.05	1.08	1.04	1.09	1.25	1.40	1.05	1.20	1.17	1.28	1.15	1.32	1.10
SPREF1-C	1.06	1.41	0.99	1.14	0.83	0.93	1.01	1.03	0.99	1.00	0.88	1.03	1.03	1.17	1.16	1.29	1.12	1.26	1.06
Base-64KB	1.02	1.08	1.04	1.11	1.02	1.03	1.07	1.11	1.10	1.15	1.03	1.04	1.00	1.06	1.01	1.02	1.00	1.02	1.03

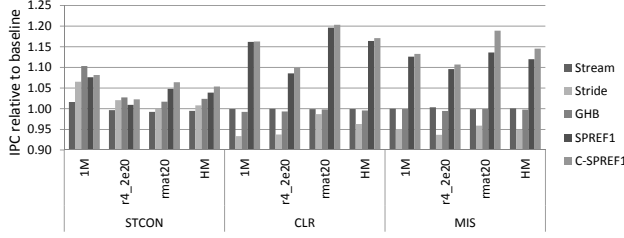


Figure 10: IPC improvements - STCON, CLR, MIS

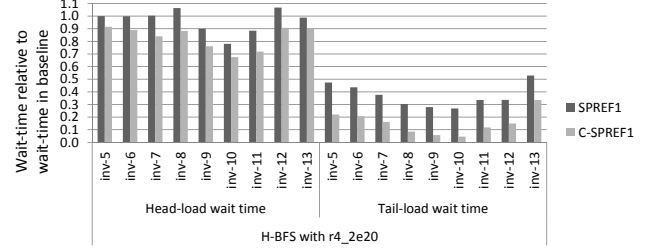


Figure 11: Wait times - H-BFS with r4_2e20 input

previously proposed mechanisms either show no performance improvement or minor performance improvement for most of the evaluated kernels, SPREF1 and C-SPREF1 show significant performance improvement for different kernels with different inputs. The average performance improvement shown by SPREF1 and the other mechanisms across the evaluated kernels is shown in Table 7. On average, the improvement provided by SPREF1, C-SPREF1, SPREF and C-SPREF across all kernel invocations of the evaluated kernels for all inputs are 8%, 9%, 8% and 10%.

5.2. Analysis of performance improvements

5.2.1 Stride, Stream and GPH prefetchers

Previously proposed mechanisms have difficulty in learning the addresses to be prefetched. For most kernel invocations, Stride prefetcher sends out very few prefetch requests because most loads have zero stride (head-loads) or random strides and the Stride prefetcher is unable to learn any patterns. On the otherhand, Stream and GHB are able to detect some patterns and send out significant prefetches for most kernel invocations. However, the fraction of useful prefetches is very small, especially for Stream, and hence we are unable to see much benefit from these mechanisms. Also for Stream and GHB prefetchers, most of the prefetches get evicted without being used.

5.2.2 SPREF1 and variants

SPREF1 and related prefetchers mainly improve performance by reducing the wait time for load-pairs, especially the dependent load. Here, wait time refers to the duration for which the instruction dependent on the load waits for

the load to complete. The wait times for tail-loads are typically longer than the wait times for the head-loads and multithreading hides some of the wait time of a load. Due to the reduction in wait times provided by SPREF1 (and variants) the number of core idle cycles are also reduced resulting in performance improvement. As an example, Figure 11 shows the wait times relative to the wait times for the baseline for the head-load and the tail-load for H-BFS with r4_2e20 input for different kernel invocations during the same application run. While the wait time for the head-load has reduced in some cases and increased in others, the wait time for the tail-load has reduced significantly and this provides performance benefit. Below we briefly analyze the performance improvement shown by the different kernels while analyzing the performance of H-BFS, especially H-BFS with r4_2e20 in detail.

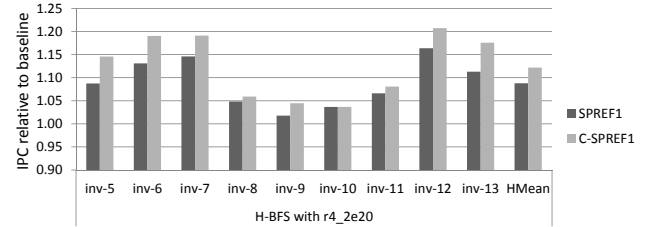


Figure 12: IPC improvements - H-BFS with r4_2e20 input

H-BFS H-BFS has one load-pair and across all inputs used, SPREF1 and C-SPREF1 show an average performance improvement of 12% and 14% for this kernel. In H-BFS and other kernels a notion of active/enabled vertex is used (except LS-BFS and LS-SSSP, in these kernels all

vertices are active in every kernel invocation); in a kernel invocation, a vertex processes its neighbors only if the vertex is active/enabled. In many iterations (especially iterations at the start and at the end of the algorithm) often only a few vertices are active, these invocations tend to show low improvement since there are only a few opportunities for prefetching (this is applicable to other kernels as well).

Figure 12 shows the performance improvement provided by SPREF1 and C-SPREF1 for H-BFS with $r4_2e20$ input across different kernel invocations in the same application run. Since the vertices that are active during each step (invocation) of BFS are different, the performance improvement also varies from one invocation to another. In general the performance improvement increases with the number of active vertices and the number of neighbors per active vertices. However, when the number of active vertices is high, sometimes performance improvement could be reduced due to contention in the memory hierarchy as in the case of invocations 9, 10 and 11 in Figure 12.

Figure 12 also shows that C-SPREF1 provides slightly higher performance improvement than SPREF1. Since C-SPREF1 does not have the overhead of detecting target loads and does not take an additional loop iteration to determine loop iteration counts before inserting prefetches, it has additional opportunities for inserting prefetches and thus can show more benefit than SPREF1.

H-SSSP This kernel has only head-loads and no tail-loads. One of the head-loads has a dependent memory access similar to a tail-load, however, the memory location is read and updated via an atomic instruction, hence the location cannot be prefetched into a register or the L1 cache and we are unable to improve performance significantly.⁶ For this kernel we only prefetch the head-loads into registers.

H-MST In addition to the detected load-pair, this kernel has several other loads and one of them is dependent on the tail-load in the identified load-pair. Reducing the wait time of only the load pair without reducing the wait time of other loads does not improve performance much and thus we see small performance gains with SPREF1 and C-SPREF1.

LS-BFS and LS-SSSP Similar to H-SSSP, these kernels have only a head-load and the dependent memory access is performed atomically, hence the only the head-load can be prefetched into registers (and cache) and this does not provide much benefit.

LS-MST, STCON, CLR, MIS All these kernels have a load pair and the kernels are similar to H-BFS. In each kernel invocation a set of active/enabled vertices traverse their neighbor list. Like H-BFS these kernels tend to show good benefit except for STCON which terminate quickly within a few iterations for the evaluated inputs and hence does not have many active vertices during each invocation and therefore shows slightly lower performance improvement.

⁶atomic instructions update memory locations directly in the L2 cache

5.3. Compiler-assisted mechanism and dynamic prefetch distance

Compiler-assisted mechanism Though the compiler-assisted mechanisms (C-SPREF1 and C-SPREF) perform slightly better than the hardware-only mechanisms (SPREF1 and SPREF) the difference between them is not very significant (Table 7). For the hardware-only mechanisms, a warp must execute at least one iteration before the iteration counts of the warp's threads can be determined. Due to this overhead, SPREF1 and SPREF often have fewer opportunities for prefetching than their compiler-assisted counterparts. For the same reason, SPREF is likely to execute with a smaller dynamic prefetch distance than C-SPREF.

Dynamic prefetch distance Although using a dynamic prefetch distance does not improve performance by much on average, there are instances which show improvement as shown by the increase in the maximum IPC improvement for H-BFS and LS-MST (SPREF vs SPREF1 and C-SPREF vs C-SPREF1).

5.4. Benefit from prefetching into registers

We analyze the performance improvement due to prefetching into spare registers in addition to the cache. The row labeled SPREF1-C shows the performance improvement due to a mechanism that is the same as the proposed hardware mechanism, but does not insert the tail prefetches into a spare register but inserts them only into the cache. The head-loads would have to be inserted into spare registers since they are needed to calculate the prefetch address for the tail-loads. For kernels that do not have tail-loads, the head-loads are prefetched but inserted into the cache only. The performance of SPREF1-C in comparison to SPREF1 is varied. For kernels such as STCON, CLR and MIS, the performance of SPREF1-C is similar to SPREF1. But for kernels such as H-BFS, H-MST and LS-MST there is severe performance degradation for several kernel instances.

As mentioned, inserting prefetched data into cache only has a downside - prefetched data could get evicted from L1 requiring additional memory requests to the lower levels of the memory hierarchy, while the L1 data cache will have more requests to contend with (both prefetch and demand requests) when compared to SPREF1.

5.5. No prefetching with a larger cache

We increase the cache size for the baseline to 64KB for all kernels to see whether a baseline with a larger cache can provide the same performance as SPREF1 on a GPU with a normal sized cache (48KB/16KB for Fermi architecture depending on the kernel). For all kernels, combining the data cache and the consumed spare registers, SPREF1 does not consume more than 64KB of space. The row labeled Base-64KB in Table 7 shows the IPC improvement obtained by increasing the cache size to 64KB. While there is an

improvement in performance for most kernels when compared to the baseline, the improvement is considerable for LS-BFS, and LS-SSSP. Compared to SPREF1, Base-64KB performs better only for H-SSSP, LS-BFS and LS-SSSP. As mentioned above, in case of these kernels, there are only head-loads and hence SPREF1 cannot improve their performance significantly. In case of LS-SSSP and LS-BFS, SPREF1 (and baseline) uses only a 16KB data cache, because these kernels use 48KB of the 64KB on-chip memory as Shared Memory. This results in a 64KB data cache providing significant additional cache space over what is used by SPREF1 and thus helps obtain a much higher performance. However, on average, our proposed mechanism shows better performance than a GPU with a larger cache by providing effective prefetching mechanisms.

We also evaluate SPREF1 on a machine with a unified on-chip storage of 192KB (same on-chip storage as Fermi). In such a machine the register file, Shared Memory and data cache all share the same physical storage. After allocating registers and Shared Memory as per the kernel requirements, the rest of the on-chip storage is allocated as data cache. In case of SPREF1 we allocate registers plus spare registers for prefetching and Shared Memory as per kernel requirements before allocating the remaining space as data cache. For such a machine configuration, SPREF1 performs better than no prefetching by 7% on average while performing slightly worse for H-MST and H-SSSP. Thus, though SPREF1 duplicates some data it makes effective use of on-chip storage.

6. Related work

6.1. Hardware GPU Prefetching

Very few GPU specific hardware prefetching mechanisms have been proposed thus far. Lee et al. [27] propose Many-Thread Aware Prefetcher (MTAP) for GPGPU applications. MTAP uses a table based mechanism to detect strides in thread accesses and does both intra-warp and inter-warp prefetching. Our mechanism uses a tag based approach for detecting target loads and their properties and does intra-thread prefetching only. When generating prefetches MTAP assumes that all threads are active in a warp, which is not always true for graph algorithms. Also, in case of graph algorithms, MTAP would be unlikely to detect any inter-warp strides due to data dependent memory accesses. The intra-warp component of MTAP may detect strided accesses by the head-loads, but prefetching only the head-loads is beneficial.

6.2. Software GPU Prefetching

Ryoo et al. [40] show that binding prefetches can improve performance, and also propose a set of metrics to predict the performance improvement when different program optimizations are applied to GPGPU benchmarks. Their

work does not propose any prefetching mechanisms for GPGPUs.

Yan et al. [50] propose a compiler which does register based prefetching that is similar to our mechanism. While their approach is completely software based, our approach can be entirely hardware-based. While the compiler-based prefetching mechanism has to use a fixed prefetch distance, our mechanism uses a dynamic prefetch distance which can help improve performance. In addition, the compiler based mechanism requires additional registers to be available for all warps to be able to insert prefetches. Our mechanism can be modified to do prefetching for only a subset of the warps when insufficient spare registers are available by allocating spare registers to only some of the warps.

Using CPUs to prefetch for GPUs by doing pre-execution has also been proposed [51]. While this mechanism can potentially prefetch all memory accesses, it can work only on fused CPU-GPU architectures. On the other hand, our mechanism prefetches only a subset of the accesses, but does not require the assistance of a CPU.

6.3. Pointer-based Prefetching

Several pointer-based hardware and software prefetching mechanisms [30, 39, 38, 9, 11, 14, 49] have been proposed for CPUs. Our mechanism can also be considered to be doing pointer-based prefetching - in a load-pair, the head load points to the index to be accessed by the tail-load. Some of these mechanisms augment the data structure being prefetched with additional fields (jump-pointers) while our mechanism does not modify the prefetched array. In addition, these mechanisms place prefetched data into cache and not registers and are likely to consume additional bandwidth due to eviction of prefetched data.

Prefetching via pre-execution by newly spawned helper threads [29, 41] has also been proposed for pointer intensive applications. These mechanisms also suffer from early evictions. Also, such mechanisms require compiler assistance, have high overhead due to helper thread creation and re-execution of most instructions. Our mechanism also results in execution of additional instructions but by targeting only the common graph algorithm patterns, the cost of additional instructions is kept low.

A hardware-software cooperative prefetching scheme which uses hint bits generated by the compiler to regulate an aggressive hardware prefetching engine has also been proposed [48]. In a version of our mechanism the compiler identifies the target loads and their properties, instead of providing hints about different loads. Also, we propose a prefetching mechanism rather than a mechanism to reduce bandwidth consumption by prefetches.

6.4. Irregular memory accesses in GPUs

Not much research has been done for irregular applications in GPUs unlike CPU applications. Run-ahead of ex-

ecution of threads with cache hits in a memory divergent warp has been proposed to issue memory requests ahead of time [44, 31]. Similar to these mechanisms our mechanism issues memory requests ahead of time, however our mechanism issues memory requests ahead of time for all threads in a warp and does not do run-ahead execution. These mechanisms require complex hardware to support divergence and to merge split warps as well.

6.5. Work related to register files in GPUs

Recently, there have been some proposals related to unallocated registers. Abdel-Majeed et al. proposed to reduce power consumption by turning off unallocated registers [3]. Gebhart et al. proposed several solutions to utilize register files such as using using register file caches [17, 18] or modified register file hierarchies [18, 19]. On the contrary, our proposal utilizes unallocated registers to store prefetched data. Furthermore, the remaining unallocated registers can be also still turned off to save power.

7. Conclusion

In this work, we take advantage of unallocated registers in GPUs to design a prefetching mechanism for graph algorithms which tend to be memory intensive with data dependent memory accesses. Using a tag-based hardware mechanism, we identify target load-pairs that are in a loop for prefetching. Once the target loads are identified, the mechanism injects instructions to prefetch data accessed in subsequent loop iterations into unallocated registers. The loads whose addresses have been prefetched are converted into mov instructions to use the prefetched data. The mechanism has the benefit that even if the prefetched data gets evicted from cache, the data would still be available in the registers. Also, our mechanism can vary the prefetch distance on a per loop invocation basis and we also propose a compiler-assisted mechanism besides the hardware only mechanism. The hardware-only mechanism provides an average benefit of 8% and a maximum benefit of 50% across nine memory intensive graph kernels with different inputs. The numbers for the compiler-assisted prefetching mechanism are 10% (average) and 51% (maximum). In future, we will apply the idea of prefetching into spare registers for applications with other memory access patterns.

8. Acknowledgements

We gratefully acknowledge the support of National Science Foundation (NSF) CAREER CCF 1054830, NSF CCF 1337177 and Sandia National Laboratories and the hardware donations from NVIDIA. We would like to thank other HPArch members and the anonymous reviewers for their comments and suggestions. Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect those of NSF or Sandia National Laboratories.

References

- [1] HYNIX GDDR5 SGRAM. [hynix.co.kr/inc/pdfDownload.jsp?path=/datasheet/pdf/graphics/H5GQ1H24AFR\(Rev1.0\).pdf](http://hynix.co.kr/inc/pdfDownload.jsp?path=/datasheet/pdf/graphics/H5GQ1H24AFR(Rev1.0).pdf).
- [2] MacSim. code.google.com/p/macsim/.
- [3] M. Abdel-Majeed and M. Annavaram. Warped register file: A power efficient register file for gpgpus. In *Proceedings of the 19th IEEE International Symposium on High Performance Computer Architecture*, HPCA '13, 2013.
- [4] N. Brunie, S. Collange, and G. Diamos. Simultaneous branch and warp interweaving for sustained gpu performance. In *Proceedings of the 39th Annual International Symposium on Computer Architecture*, ISCA '12, 2012.
- [5] M. Burtcher, R. Nasre, and K. Pingali. A quantitative study of irregular programs on gpus. In *Workload Characterization (IISWC), 2012 IEEE International Symposium on*, pages 141–151, 2012.
- [6] D. Chakrabarti, Y. Zhan, and C. Faloutsos. R-mat: A recursive model for graph mining. In *In SDM*, 2004.
- [7] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron. Pannotia: Understanding irregular gpgpu graph applications. 2013.
- [8] J. Cohen and P. Castonguay. Efficient graph matching and coloring on the gpu. developer.download.nvidia.com/GTC/PDF/GTC2012/PresentationPDF/S0332-GTC2012-Graph-Coloring-GPU.pdf.
- [9] J. D. Collins, S. Sair, B. Calder, and D. M. Tullsen. Pointer cache assisted prefetching. In *MICRO-35*, pages 62–73, Los Alamitos, CA, USA, 2002. IEEE Computer Society Press.
- [10] R. Cooksey. *Content-Sensitive Data Prefetching*. PhD thesis, University of Colorado, Boulder, 2002.
- [11] R. Cooksey, S. Jourdan, and D. Grunwald. A stateless, content-directed data prefetching mechanism. In *Proc. of the 10th Int'l. conference on Architectural support for programming languages and operating systems*, pages 279–290, New York, NY, USA, 2002. ACM.
- [12] G. Diamos, B. Ashbaugh, S. Maiyuran, A. Kerr, H. Wu, and S. Yalamanchili. Simd re-convergence at thread frontiers. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-44 '11, 2011.
- [13] G. Diamos, A. Kerr, S. Yalamanchili, and N. Clark. Ocelot: A dynamic compiler for bulk-synchronous applications in heterogeneous systems. In *PACT-19*, 2010.
- [14] E. Ebrahimi, O. Mutlu, and Y. N. Patt. Techniques for bandwidth-efficient prefetching of linked data structures in hybrid prefetching systems. In *HPCA-15*, pages 7–17, Washington, DC, USA, 2009. IEEE Computer Society.
- [15] W. W. L. Fung and T. M. Aamodt. Thread block compaction for efficient simt control flow. In *Proceedings of the 2011 IEEE 17th International Symposium on High Performance Computer Architecture*, HPCA '11, 2011.
- [16] W. W. L. Fung, I. Sham, G. Yuan, and T. M. Aamodt. Dynamic warp formation and scheduling for efficient gpu control flow. In *MICRO*, 2007.
- [17] M. Gebhart, D. R. Johnson, D. Tarjan, S. W. Keckler, W. J. Dally, E. Lindholm, and K. Skadron. Energy-efficient mechanisms for managing thread context in throughput processors. In *Proceedings of the 38th annual international symposium on Computer architecture*, ISCA '11, 2011.
- [18] M. Gebhart, S. W. Keckler, and W. J. Dally. A compile-time managed multi-level register file hierarchy. In *Proceedings*

- of the 44th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-44 '11, 2011.
- [19] M. Gebhart, S. W. Keckler, B. Khailany, R. Krashinsky, and W. J. Dally. Unifying primary cache, scratch, and register file memories in a throughput processor. In *Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '12, 2012.
 - [20] P. Harish and P. J. Narayanan. Accelerating large graph algorithms on the gpu using cuda. In *Proceedings of the 14th international conference on High performance computing*, HiPC'07, 2007.
 - [21] P. Harish, V. Vineet, and P. J. Narayanan. Large graph algorithms for massively multithreaded architectures. Technical Report IIIT/TR/2009/74, International Institute of Information Technology, Hyderabad, INDIA, 2009.
 - [22] S. Hong, S. K. Kim, T. Oguntebi, and K. Olukotun. Accelerating cuda graph algorithms at maximum warp. In *Proceedings of the 16th ACM symposium on Principles and practice of parallel programming*, PPoPP '11, 2011.
 - [23] S. Hong, T. Oguntebi, and K. Olukotun. Efficient parallel graph exploration on multi-core cpu and gpu. In *Parallel Architectures and Compilation Techniques (PACT), 2011 International Conference on*, pages 78–88, 2011.
 - [24] S. Iacobovici, L. Spracklen, S. Kadambi, Y. Chou, and S. G. Abraham. Effective stream-based and execution-based data prefetching. In *ICS-18*, pages 1–11, 2004.
 - [25] K. J. Nesbit and J. E. Smith. Data cache prefetching using a global history buffer. In *HPCA-10*, pages 96–105, Washington, DC, USA, 2004. IEEE Computer Society.
 - [26] D. Joseph and D. Grunwald. Prefetching using Markov predictors. In *ISCA-19*, pages 252–263, New York, NY, USA, 1997. ACM.
 - [27] J. Lee, N. B. Lakshminarayana, H. Kim, and R. Vuduc. Many-thread aware prefetching for gpgpus. In *MICRO-43*, 2010.
 - [28] M. Luby. A simple parallel algorithm for the maximal independent set problem. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC '85, 1985.
 - [29] C.-K. Luk. Tolerating memory latency through software-controlled pre-execution in simultaneous multithreading processors. In *ISCA-23*, pages 40–51, New York, NY, USA, 2001. ACM.
 - [30] C.-K. Luk and T. C. Mowry. Compiler-based prefetching for recursive data structures. In *Proc. of the 7th Int'l. conference on Architectural support for programming languages and operating systems*, pages 222–233, New York, NY, USA, 1996. ACM.
 - [31] J. Meng, D. Tarjan, and K. Skadron. Dynamic warp subdivision for integrated branch and memory divergence tolerance. In *ISCA-32*, 2010.
 - [32] D. Merrill, M. Garland, and A. Grimshaw. Scalable gpu graph traversal. In *Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, PPoPP '12, 2012.
 - [33] K. J. Nesbit, A. S. Dhodapkar, and J. E. Smith. AC/DC: An adaptive data cache prefetcher. In *PACT-13*, pages 135–145, Washington, DC, USA, 2004. IEEE Computer Society.
 - [34] NVIDIA. Fermi: Nvidia's next generation cuda compute architecture. <http://www.nvidia.com/fermi>.
 - [35] NVIDIA. Nvidia's next generation cuda compute architecture: Kepler gk110. <http://www.nvidia.com/content/PDF/kepler/NVIDIA-Kepler-GK110-Architecture-Whitepaper.pdf>.
 - [36] NVIDIA Corporation. *CUDA Programming Guide*.
 - [37] M. Rhu and M. Erez. Capri: prediction of compaction-adequacy for handling control-divergence in gpgpu architectures. In *Proceedings of the 39th Annual International Symposium on Computer Architecture*, ISCA '12, 2012.
 - [38] A. Roth, A. Moshovos, and G. S. Sohi. Dependence based prefetching for linked data structures. In *Proc. of the 8th Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems*, 1998.
 - [39] A. Roth and G. S. Sohi. Effective jump-pointer prefetching for linked data structures. In *ISCA-21*, pages 111–121, Washington, DC, USA, 1999. IEEE Computer Society.
 - [40] S. Ryoo, C. I. Rodrigues, S. S. Stone, S. S. Baghsorkhi, S.-Z. Ueng, J. A. Stratton, and W. mei W. Hwu. Program optimization space pruning for a multithreaded gpu. In *CGO-6*, pages 195–204, 2008.
 - [41] R. H. Saavedra and D. Park. Improving the effectiveness of software prefetching with adaptive execution. In *Proc. of the 5th Int'l. Conf. on Parallel Architectures and Compilation Techniques*, pages 68–78, Washington, DC, USA, 1996. IEEE Computer Society.
 - [42] B. Sinharoy, R. N. Kalla, J. M. Tendler, R. J. Eickemeyer, and J. B. Joyner. Power5 system microarchitecture. *IBM Journal of Research and Development*, 49(4-5):505–522, 2005.
 - [43] J. Soman, K. Kishore, and P. J. Narayanan. A fast gpu algorithm for graph connectivity. In *Parallel Distributed Processing, Workshops and Phd Forum (IPDPSW), 2010 IEEE International Symposium on*, pages 1–8, 2010.
 - [44] D. Tarjan, J. Meng, and K. Skadron. Increasing memory miss tolerance for simd cores. In *SC*, 2009.
 - [45] J. Tendler, S. Dodson, S. Fields, H. Le, and B. Sinharoy. POWER4 system microarchitecture. *IBM Journal of Research and Development*, 46(1):5–25, 2002.
 - [46] The IMPACT Research Group, UIUC. Parboil benchmark suite. <http://impact.crhc.illinois.edu/parboil.php>.
 - [47] V. Vineet, P. Harish, S. Patidar, and P. J. Narayanan. Fast minimum spanning tree for large graphs on the gpu. In *Proceedings of the Conference on High Performance Graphics 2009*, HPG '09, 2009.
 - [48] Z. Wang, D. Burger, K. S. McKinley, S. K. Reinhardt, and C. C. Weems. Guided region prefetching: a cooperative hardware/software approach. In *ISCA-25*, pages 388–398, New York, NY, USA, 2003. ACM.
 - [49] Y. Wu. Efficient discovery of regular stride patterns in irregular programs and its use in compiler prefetching. In *Proc. of the ACM SIGPLAN 2002 Conf. on Programming Language Design and Implementation*, pages 210–221, New York, NY, USA, 2002. ACM.
 - [50] Y. Yang, P. Xiang, J. Kong, and H. Zhou. A gpgpu compiler for memory optimization and parallelism management. In *Proc. of the ACM SIGPLAN 2010 Conf. on Programming Language Design and Implementation*, 2010.
 - [51] Y. Yang, P. Xiang, M. Mantor, and H. Zhou. Cpu-assisted gpgpu on fused cpu-gpu architectures. In *High Performance Computer Architecture (HPCA), 2012 IEEE 18th International Symposium on*, 2012.